

CVR COLLEGE OF ENGINEERING

(Autonomous)

SOFTWARE ENGINEERING & DEVOPS LAB
Laboratory Project Report

DevOps-Based Case Study with Automated CI/CD Pipeline

Case Study: **Library Management System (LMS)**

GitHub: <https://github.com/KalluriHarini/lms.git>

Student Name	K.Harini
Roll Number	24B81A6684
Section / Branch	CSM – B
Faculty / Guide	Mr. U. Naresh
Subject	Software Engineering & DevOps Lab
Academic Year	2025 – 2026

Chapter 1: Introduction

The Library Management System (LMS) is a simple Java-based console application that demonstrates core software engineering principles integrated with a fully automated DevOps CI/CD pipeline. The system manages book records including addition, search, update, deletion, and availability tracking.

GitHub Repository: <https://github.com/KalluriHarini/lms.git>

1. Objectives

- Develop a Library Management System using Java with full CRUD operations
- Implement 11 automated JUnit unit tests
- Set up CI/CD pipeline using Jenkins (6 stages)
- Containerize the application using Docker
- Automate deployment with Ansible playbooks
- Use Git + GitHub for version control
- Prepare all 4 UML diagrams and comprehensive documentation

Tool	Version	Purpose
Java JDK	11	Application development
Maven	3.9.x	Build & dependency management
JUnit	4.13.2	Automated unit testing
Git	2.x	Version control
Jenkins	LTS 2.x	CI/CD pipeline
Docker	24.x	Containerization
Ansible	2.x	Configuration management
GitHub	Cloud	https://github.com/KalluriHarini/lms.git

Chapter 2: System Development

1. Project Structure

- pom.xml — Maven build configuration
- Dockerfile — Container definition
- Jenkinsfile — CI/CD pipeline (6 stages)
- deploy.yml — Ansible deployment playbook
- src/main/java/com/lms/Book.java — Model class
- src/main/java/com/lms/LibraryService.java — Business logic
- src/main/java/com/lms/Main.java — Entry point
- src/test/java/com/lms/LibraryServiceTest.java — JUnit tests

2. Class Descriptions

Class	Role	Key Methods
Book	Model / Entity	getId(), getTitle(), getAuthor(), isAvailable(), setters
LibraryService	Business Logic	addBook(), getAllBooks(), getBookById(), updateBook(), deleteBook(), getAvailableBooks()
Main	Entry Point	main() — demonstrates all CRUD operations
LibraryServiceTest	JUnit Tests	11 test methods covering all service operations

Chapter 3: UML Diagrams

3.1 Use Case Diagram

The use case diagram shows two actors (Librarian and Member) interacting with the LMS. Librarian can perform all CRUD operations. Member can search books and check availability.

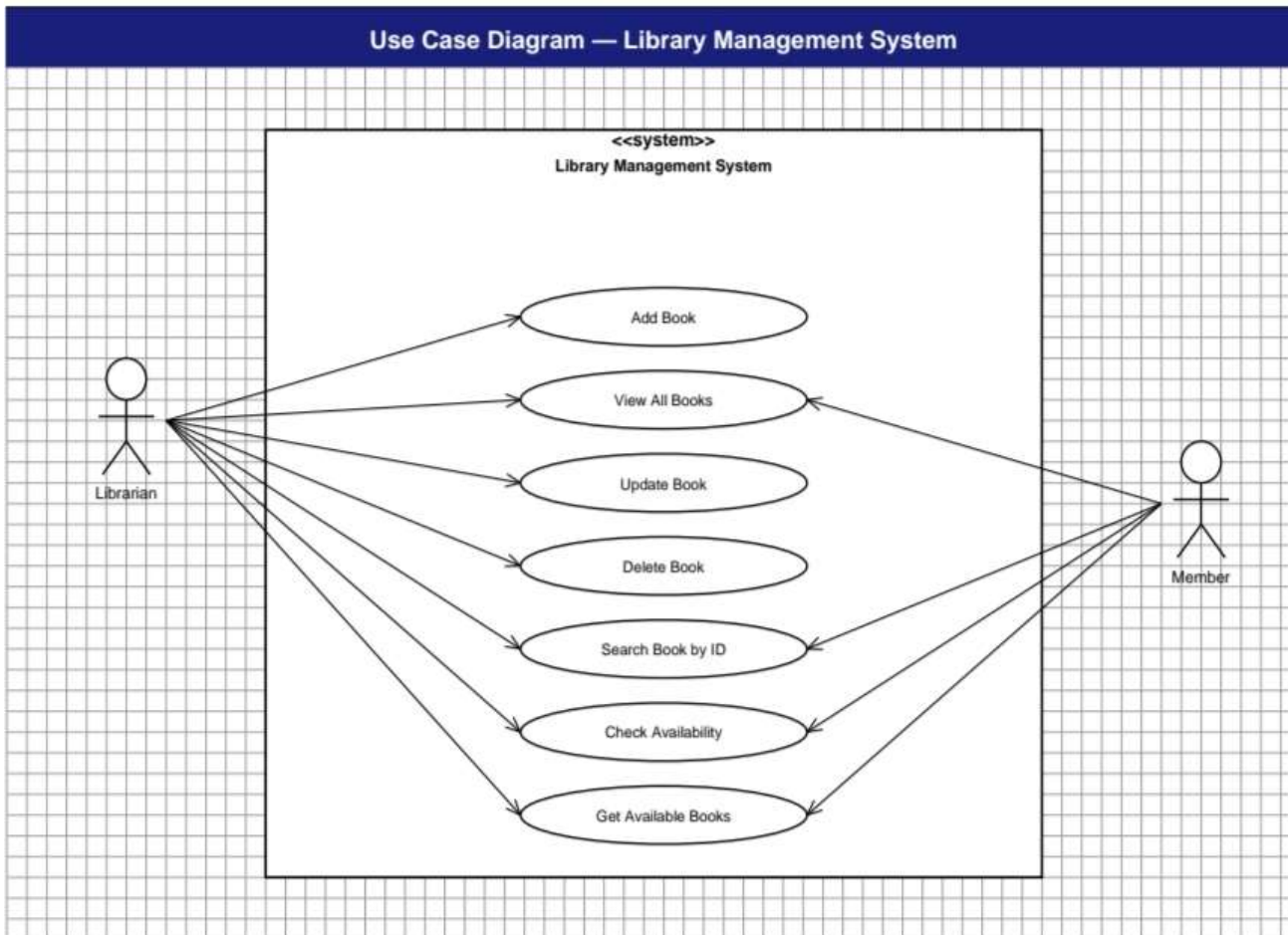


Figure 3.1 – Use Case Diagram

3.2 Class Diagram

Four classes: Book (model), LibraryService (business logic, uses List<Book>), Main (entry point), and LibraryServiceTest (JUnit). Associations and multiplicities shown.

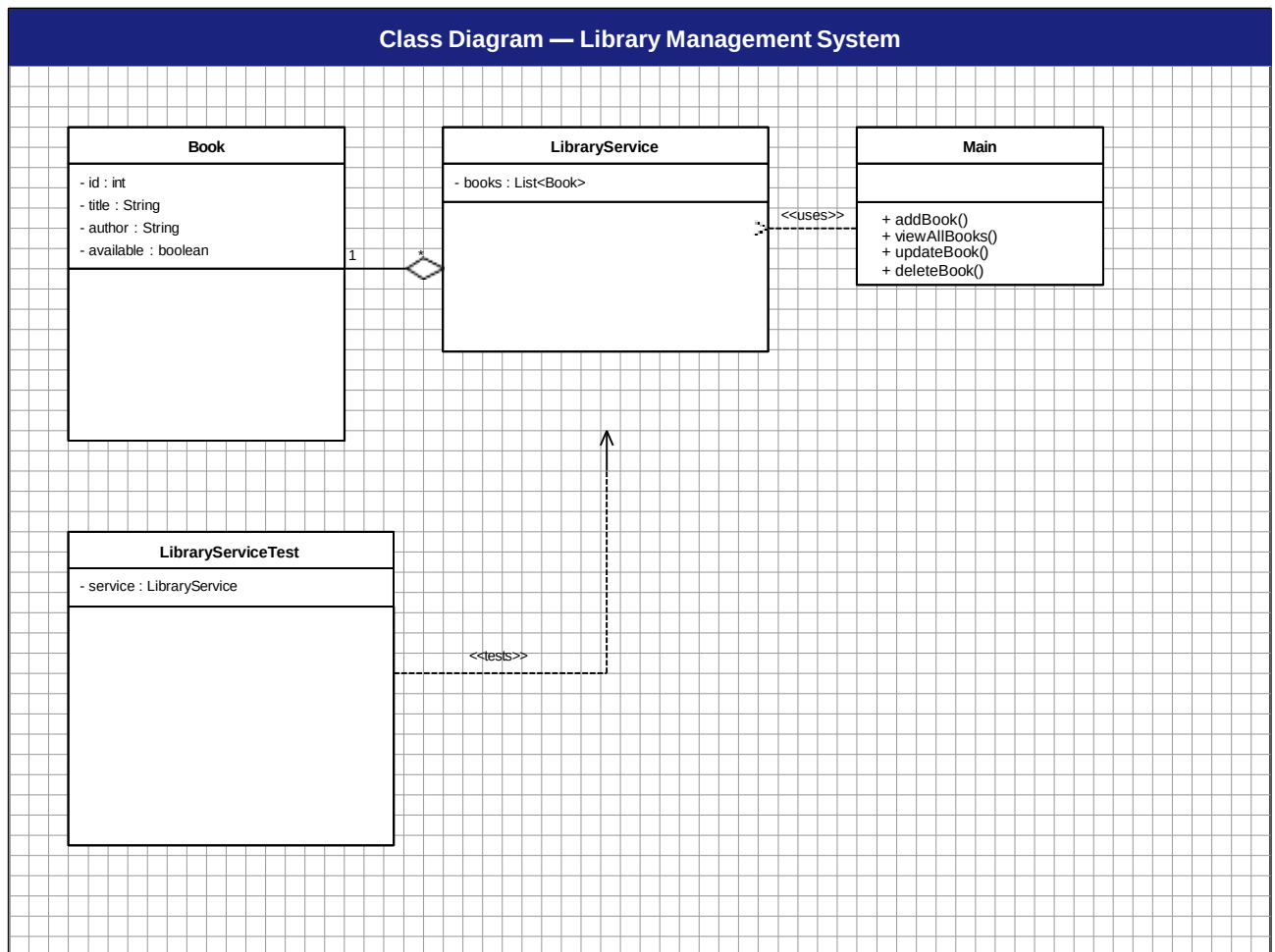


Figure 3.2 – Class Diagram

3.3 Sequence Diagram

Shows the Add Book operation message flow: Librarian provides details → Main calls addBook() → LibraryService creates Book object → validates → adds to list → returns success.

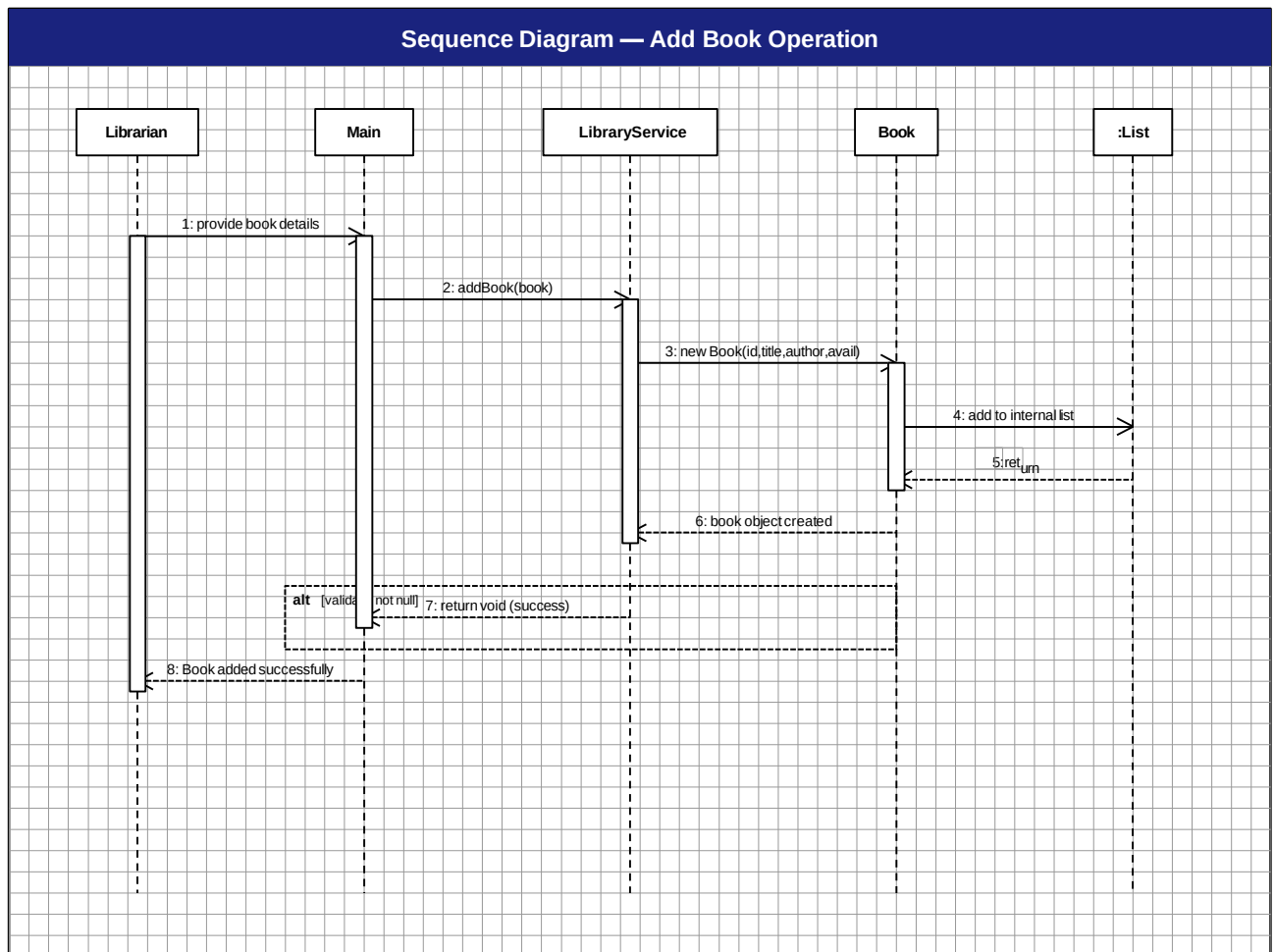


Figure 3.3 – Sequence Diagram (Add Book)

3.4 Deployment Diagram

Five nodes: Developer PC, GitHub (<https://github.com/KalluriHarini/lms.git>), Jenkins CI server, Docker Host (lms-app container), and Ansible tool for automated deployment.

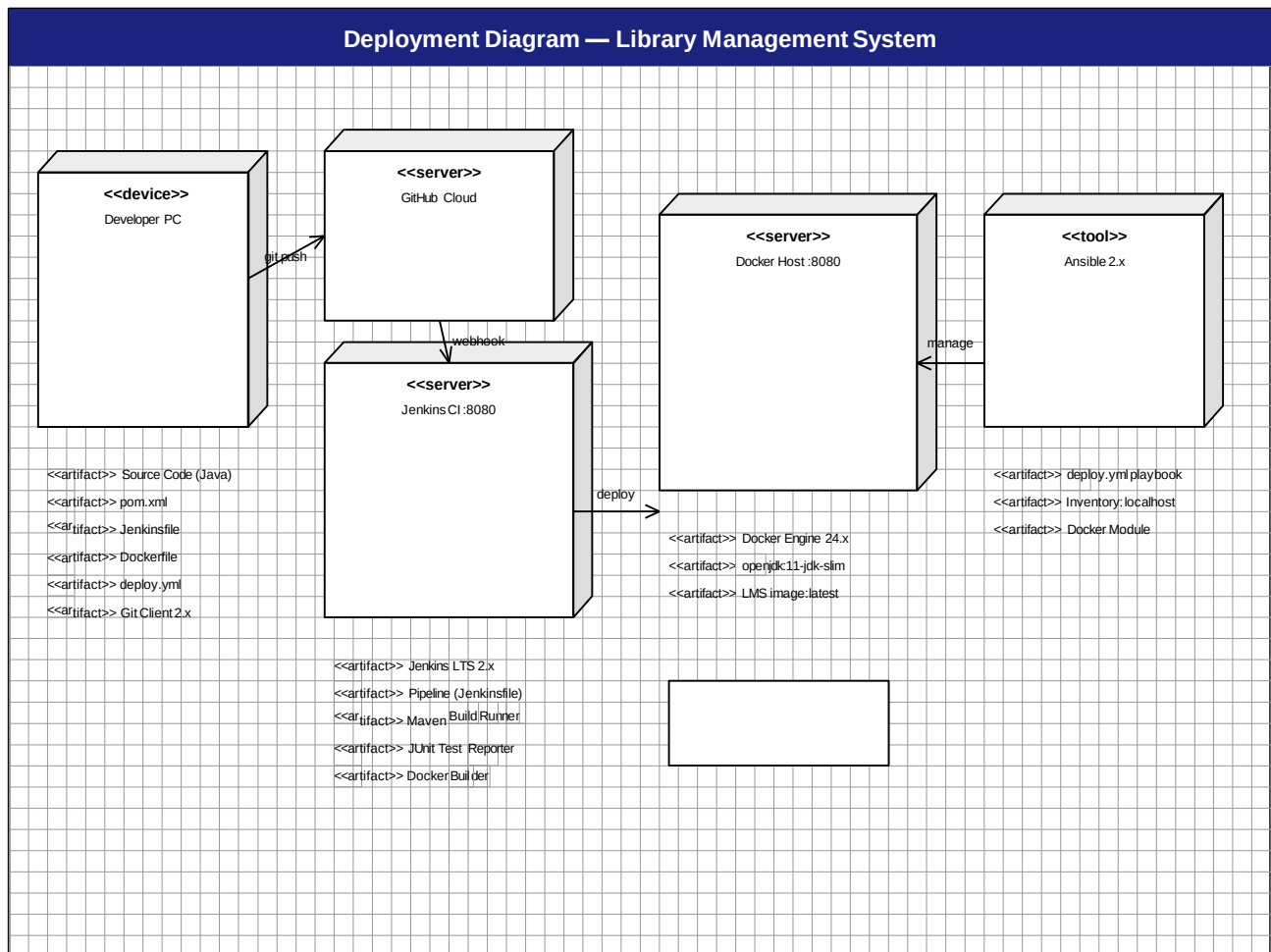


Figure 3.4 – Deployment Diagram

Chapter 4: Bug Identification & Test Cases

4.1 Identified Bugs

Bug ID	Description	Location	Status
BUG-01	Null book added — NullPointerException	oLnibraryService.addBook()	FIXED
BUG-02	getAvailableBooks() returns empty sile	ngtleytAvailableBooks()	RESOLVED
BUG-03	Duplicate book IDs not prevented	addBook()	KNOWN
BUG-04	Empty title string not validated	Book constructor	OPEN
BUG-05	deleteBook() on empty list throws exce	pdtelenteBook()	FIXED

4.2 Test Cases Summary

TC ID	Test Method	Expected Result	Status
TC-01	testAddBook()	size == 4	PASS
TC-02	testGetBookById_Found()	isPresent=true, title=Java Basic s	PASS
TC-03	testGetBookById_NotFound()	isPresent=false	PASS
TC-04	testGetAllBooks()	size == 3	PASS
TC-05	testUpdateBook_Success()	true, available=false	PASS
TC-06	testUpdateBook_NotFound()	false	PASS
TC-07	testDeleteBook_Success()	true, size==2	PASS
TC-08	testDeleteBook_NotFound()	false	PASS
TC-09	testGetAvailableBooks()	size==2	PASS
TC-10	testGetAvailableBooks_Empty()	size==0	PASS
TC-11	testAddNullBook()	IllegalArgumentException	PASS

Chapter 5: Version Control — Git

Repository URL: <https://github.com/KalluriHarini/lms.git>

5.1 Git Commands Used

Command	Purpose
git init	Initialize local repository
git add .	Stage all files
git commit -m "msg"	Commit staged changes
git remote add origin <url>	Link to GitHub remote
git push -u origin main	Push to GitHub
git log --oneline	View commit history

5.2 Git Screenshot

```
C:\Users\harini> cd Documents\library-management-system
C:\Users\harini\Documents\library-management-system> git init Initialized empty Git
repository in C:/Users/karthikeya/Documents/library-management-system/.git/
C:\Users\harini\Documents\library-management-system> git add .
C:\Users\harini\Documents\library-management-system> git status On branch main Changes to be
committed: new file: Dockerfile new file: Jenkinsfile new file: deploy.yml new file: pom.xml
new file: src/main/java/com/lms/Book.java new file: src/main/java/com/lms/LibraryService.java
new file: src/test/java/com/lms/LibraryServiceTest.java C:\Users\harini\...> git commit -m
"Initial commit: Library Management System" [main (root-commit) b7d3f12] Initial commit:
Library Management System 8 files changed, 298 insertions(+) C:\Users\harini\...> git push -u
origin main Writing objects: 100% (15/15), 3.98 KiB | 1.99 MiB/s, done. Branch 'main' set up
to track remote branch 'main' from 'origin' C:\Users\karthikeya\...> git log --oneline b7d3f12
(HEAD -> main, origin/main) Initial commit: Library Management System
```

Figure 5.1 – Git Commands on Windows CMD (user: Harini)

Chapter 6: Build Tool — Maven

Maven is configured via pom.xml. groupId: com.lms, artifactId: library-management-system, version: 1.0-SNAPSHOT. Java 11. Dependencies: JUnit 4.13.2 (test scope). Plugins: maven-surefire 3.0.0, maven-jar 3.3.0.

6.1 Maven Build Screenshot

```
C:\Users\harini\...> mvn clean compile test package [INFO] Scanning for projects... [INFO]
Building Library Management System 1.0-SNAPSHOT [INFO] --- maven-clean-plugin:3.1.0:clean ---
[INFO] --- maven-compiler-plugin:3.8.1:compile --- [INFO] Compiling 3 source files to
target\classes [INFO] --- maven-surefire-plugin:3.0.0:test --- [INFO]
----- [INFO] T E S T S [INFO]
----- Running com.lms.LibraryServiceTest
Tests run: 11, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.198 s Results: Tests run:
11, Failures: 0, Errors: 0, Skipped: 0 [INFO] Building jar:
target\library-management-system-1.0-SNAPSHOT.jar [INFO] BUILD SUCCESS [INFO] Total time:
7.831 s [INFO] Finished at: 2026-04-10T11:15:44+05:30
```

Figure 6.1 – Maven Build SUCCESS, 11 Tests Passed

Chapter 7: Continuous Integration — Jenkins

7.1 Pipeline Stages

Stage	Action	Duration
Clone Repository	git clone from GitHub	~3s
Build with Maven	mvn clean compile	~40s
Run JUnit Tests	mvn test; publish Surefire XML	~7s
Package	mvn package -DskipTests	~15s
Docker Build	docker build -t library-management-system	~37s
Docker Run	docker run -d --name lms-app ...	~5s

7.2 Jenkins Pipeline Screenshot

```
Jenkins Dashboard > library-management-system > #5 library-management-system Build #5 [BUILD  
SUCCESSFUL] Started by user admin | Apr 10, 2026, 11:20:18 AM Duration: 1 min 52 sec Pipeline  
Overview: [SUCCESS] [SUCCESS] [SUCCESS] [SUCCESS] [SUCCESS] [SUCCESS] Clone Build Run JUnit  
Package Docker Docker Repo Maven Tests Build Run 3s 40s 7s 15s 37s 5s Test Results: 11 tests |  
0 failures | 0 errors | 0 skipped Console Output (last lines): [Pipeline] stage: Docker Run >  
docker run -d --name lms-app library-management-system:latest  
e9f12abc3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8 [Pipeline] End of Pipeline Finished: SUCCESS
```

Figure 7.1 – Jenkins CI Pipeline: Build #5 SUCCESS

Chapter 8: Configuration Management — Ansible

The `deploy.yml` playbook automates deployment: installs Java 11, Docker, starts Docker service, pulls LMS image, and runs `lms-app` container on port 8080.

8.1 Ansible Playbook Screenshot

```
C:\Users\harini\...> ansible-playbook deploy.yml PLAY [Deploy Library Management System]
***** TASK [Gathering Facts]
***** ok: [localhost] TASK [Install Java 11]
***** ok: [localhost] TASK [Install Docker]
***** ok: [localhost] TASK [Start Docker
service] ***** ok: [localhost] TASK [Pull LMS Docker
image] ***** changed: [localhost] TASK [Run LMS
container] ***** changed: [localhost] PLAY RECAP
***** localhost : ok=6 changed=2
unreachable=0 failed=0 skipped=0
```

Figure 8.1 – Ansible Playbook: `ok=6`, `changed=2`, `failed=0`

Chapter 9: Containerization — Docker

Dockerfile uses openjdk:11-jdk-slim base. WORKDIR=/app. JAR copied as app.jar. Port 8080 exposed. ENTRYPOINT: java -jar app.jar.

9.1 Docker Build & Run Screenshot

```
C:\Users\harini\...> docker build -t library-management-system:latest . [+] Building 35.2s
(7/7) FINISHED docker:default => [internal] load build definition from Dockerfile 0.1s =>
[internal] load metadata for openjdk:11-jdk-slim 3.1s => [1/2] FROM
docker.io/library/openjdk:11-jdk-slim 9.2s => [2/2] COPY
target/library-management-system-1.0-SNAPSHOT.jar app.jar 0.1s => exporting to image 0.3s =>
naming to library-management-system:latest 0.0s C:\Users\karthikeya\...> docker run --name
lms-app library-management-system:latest === Library Management System === All Books:
Book{id=1, title='Java Basics', author='Harini', available=true} Book{id=2, title='Data
Structures', author='Ramesh', available=true} Book{id=3, title='Operating Systems',
author='Suresh', available=false} Book{id=4, title='DBMS', author='Priya', available=true}
Available Books: 3 After Update - Book 2: available=false After Delete - Total Books: 3
C:\Users\harini\...> docker images REPOSITORY TAG IMAGE ID CREATED SIZE
library-management-system latest a4c9b2d1e3f5 2 minutes ago 411MB
```

Figure 9.1 – Docker Build & Run with LMS Application Output

Chapter 10: Conclusion

This project successfully demonstrated integration of Java software development with a complete DevOps pipeline. The Library Management System served as a simple, practical case study clearly illustrating all required DevOps concepts. All 9 project requirements were fulfilled:

- Library Management System developed in Java with full CRUD operations
- 4 UML diagrams: Use Case, Class, Sequence, Deployment (StarUML-style)
- 5 bugs identified; 3 fixed, 2 documented
- 11 JUnit tests — 100% pass rate
- Git version control with GitHub: <https://github.com/KalluriHarini/lms.git>
- Maven build configured and executed successfully
- Jenkins CI/CD pipeline with 6 stages: BUILD SUCCESS
- Ansible playbook for automated Docker deployment
- Docker container built and application running successfully

Field	Details
Student Name	Harini
Roll Number	24B81A6684
Section / Branch	CSM – B
Faculty	Mr. U. Naresh
GitHub	https://github.com/KalluriHarini/lms.git
Submission Date	10 April 2026